

Mini – PROJECT-4

IntelliDesk AI — Enterprise IT Helpdesk Platform

Project Overview

Build an enterprise-grade AI-powered IT Helpdesk Web Application using:

- Azure OpenAI
- Azure AI Foundry
- React
- Node.js
- Express
- MongoDB
- Microsoft Graph API
- Teams Integration
- Power Automate
- ServiceNow APIs
- Azure AI Search
- Entra ID Authentication

The platform will:

- Classify incidents using AI
 - Detect severity
 - Search troubleshooting knowledge base
 - Route tickets intelligently
 - Escalate via Teams/Outlook
 - Integrate with ServiceNow
-

Enterprise Problem Statement

Large enterprises receive thousands of IT support requests daily.

Current challenges:

- Manual ticket classification

- Incorrect ticket routing
- Delayed incident response
- Repetitive support requests
- SLA violations
- Lack of intelligent automation

Goal:

Create an AI-powered intelligent IT support platform.

Enterprise Architecture

Users



React Frontend



Node.js Backend APIs



Azure OpenAI



Azure AI Search (RAG)



ServiceNow / MongoDB



Teams & Outlook Notifications



Power Automate Workflows

Major Features

Feature	Description
AI Incident Triage	AI classifies issues
Severity Detection	Detect Low/Medium/High/Critical
Knowledge Search	RAG troubleshooting
Intelligent Routing	Route tickets automatically
Teams Escalation	Notify IT teams

Feature	Description
Outlook Notifications	Email alerts
ServiceNow Integration	Create enterprise tickets
Dashboard Analytics	Ticket insights
RBAC Security	Enterprise authentication

AI Agents

Agent	Responsibility
Incident Classifier Agent	Detect issue category
Severity Agent	Priority analysis
KB Search Agent	Troubleshooting retrieval
Escalation Agent	Teams/Email escalation
Routing Agent	Assign correct team
Governance Agent	Monitor AI activity

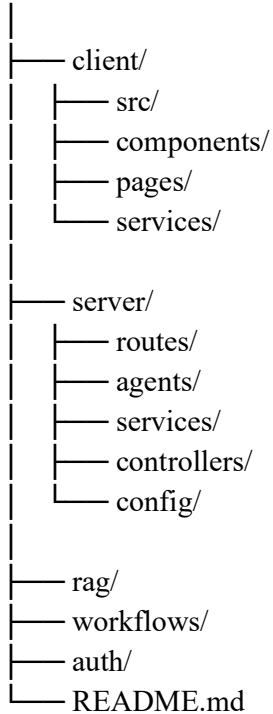
Suggested Tech Stack

Layer	Technology
Frontend	React + Tailwind
Backend	Node.js + Express
Database	MongoDB
AI	Azure OpenAI
RAG	Azure AI Search
Authentication	Azure Entra ID
Notifications	Teams + Outlook
Workflow	Power Automate

Layer	Technology
Ticketing	ServiceNow

Project Folder Structure

intellidesk-ai/



PHASE 1 — FRONTEND DEVELOPMENT

Step 1 — Create React App

```
npx create-react-app client
cd client
npm install axios react-router-dom tailwindcss
```

Step 2 — Configure Tailwind

```
npm install -D tailwindcss
npx tailwindcss init
```

Step 3 — Main App UI

App.js

```

import React from "react";
import IncidentPage from "../pages/IncidentPage";

function App() {
  return (
    <div className="bg-gray-100 min-h-screen">
      <h1 className="text-4xl font-bold text-center p-5">
        IntelliDesk AI
      </h1>

      <IncidentPage />
    </div>
  );
}

export default App;

```

Step 4 — Incident Submission UI

IncidentPage.js

```

import React, { useState } from "react";
import axios from "axios";

function IncidentPage() {

  const [incident, setIncident] = useState("");
  const [response, setResponse] = useState(null);

  const submitIncident = async () => {

    const res = await axios.post(
      "http://localhost:5000/api/incident",
      {
        incident
      }
    );

    setResponse(res.data);
  };

  return (
    <div className="p-10">

```

```

<textarea
  rows="6"
  className="w-full border p-3"
  placeholder="Describe your IT issue..."
  onChange={(e) => setIncident(e.target.value)}
/>

<button
  onClick={submitIncident}
  className="bg-blue-600 text-white px-5 py-2 mt-5"
>
  Submit Incident
</button>

{
  response && (
    <div className="bg-white p-5 mt-5 rounded shadow">

      <h2 className="text-2xl font-bold">
        AI Analysis
      </h2>

      <p>
        <strong>Category:</strong>
        {response.category}
      </p>

      <p>
        <strong>Severity:</strong>
        {response.severity}
      </p>

      <p>
        <strong>Resolution:</strong>
        {response.solution}
      </p>

    </div>
  )
}

</div>

```

```
);  
}
```

```
export default IncidentPage;
```

PHASE 2 — BACKEND DEVELOPMENT

Step 5 — Create Backend

```
mkdir server  
cd server
```

```
npm init -y
```

```
npm install express cors dotenv axios mongoose
```

Step 6 — Create Express Server

server.js

```
const express = require("express");  
const cors = require("cors");  
  
const app = express();  
  
app.use(cors());  
app.use(express.json());  
  
const incidentRoutes = require("./routes/incidentRoutes");  
  
app.use("/api", incidentRoutes);  
  
app.listen(5000, () => {  
  console.log("Server running on port 5000");  
});
```

Step 7 — Incident Route

routes/incidentRoutes.js

```
const express = require("express");  
  
const router = express.Router();
```

```
const {
  processIncident
} = require("../controllers/incidentController");

router.post("/incident", processIncident);

module.exports = router;
```

PHASE 3 — AI AGENTS IMPLEMENTATION

Step 8 — Incident Controller

controllers/incidentController.js

```
const axios = require("axios");

exports.processIncident = async (req, res) => {

  const incident = req.body.incident;

  try {

    const aiResponse = await axios.post(

      "https://YOUR_OPENAI_ENDPOINT/openai/deployments/YOUR_MODEL/chat/completions?api-version=2024-02-15-preview",

      {
        messages: [

          {
            role: "system",
            content: `
You are an enterprise IT helpdesk AI agent.

Tasks:
1. Classify incident category
2. Detect severity
3. Provide troubleshooting steps

Return JSON format:
{
```



```

    "category": "",
    "severity": "",
    "solution": ""
  }
  ,

  },

  {
    role: "user",
    content: incident
  }

],

temperature: 0.3

},

{
  headers: {
    "api-key": process.env.AZURE_OPENAI_KEY,
    "Content-Type": "application/json"
  }
}

);

const output =
  aiResponse.data.choices[0].message.content;

res.json(JSON.parse(output));

} catch (error) {

  console.log(error);

  res.status(500).json({
    error: "AI Processing Failed"
  });
}
};

```

Example Incident

My Outlook crashes repeatedly.
Teams meetings are disconnecting.
VPN access is unstable.

Example AI Response

```
{  
  "category": "Collaboration Tools",  
  "severity": "High",  
  "solution": "Restart Teams and Outlook, clear cache, verify VPN connectivity, and reinstall  
Teams if issue persists."  
}
```

PHASE 4 — KNOWLEDGE BASE SEARCH (RAG)

Step 9 — Add KB Search Agent

agents/kbAgent.js

```
async function searchKnowledgeBase(issue) {  
  
  return `  
  KB Solution:  
  - Restart Outlook  
  - Clear Teams cache  
  - Verify VPN connectivity  
  `;  
}  
  
module.exports = {  
  searchKnowledgeBase  
};
```

PHASE 5 — SEVERITY DETECTION AGENT

agents/severityAgent.js

```
function detectSeverity(issue) {  
  
  const criticalWords = [  
    "server down",  
    "vpn failure",
```

```
    "security breach"
  ];

  for (let word of criticalWords) {

    if (issue.includes(word)) {
      return "Critical";
    }
  }

  return "Medium";
}

module.exports = {
  detectSeverity
};
```

PHASE 6 — TEAMS ESCALATION

Step 10 — Teams Notification

Use Power Automate.

Trigger:

- Critical incident detected

Actions:

- Send Teams notification
 - Notify support group
 - Create ServiceNow ticket
-

Example Teams Payload

```
{
  "title": "Critical IT Incident",
  "message": "VPN outage detected for Finance Team"
}
```

PHASE 7 — SERVICE NOW INTEGRATION

Example Ticket Creation API

```
const axios = require("axios");

async function createTicket(data) {

  const response = await axios.post(

    "https://your-instance.service-now.com/api/now/table/incident",

    {
      short_description: data.issue,
      urgency: "1",
      category: data.category
    },

    {
      auth: {
        username: process.env.SN_USER,
        password: process.env.SN_PASS
      }
    }
  );

  return response.data;
}
```

PHASE 8 — DATABASE STORAGE

MongoDB Incident Model

models/Incident.js

```
const mongoose = require("mongoose");

const IncidentSchema = new mongoose.Schema({

  issue: String,
  category: String,
  severity: String,
  createdAt: {
    type: Date,
    default: Date.now
  }
})
```

```
});
```

```
module.exports =  
  mongoose.model("Incident", IncidentSchema);
```

PHASE 9 — DASHBOARD ANALYTICS

Suggested Dashboard Widgets

- Total incidents
 - Critical incidents
 - Average resolution time
 - Top issue categories
 - SLA compliance
 - AI accuracy score
-

PHASE 10 — ENTERPRISE SECURITY

Implement

- Azure Entra ID authentication
 - RBAC
 - JWT security
 - API Management
 - Managed identities
 - Audit logging
-

Suggested Enterprise Enhancements

Add:

- Multi-agent orchestration
- AI memory
- Stateful workflows
- AI governance dashboard
- Prompt injection protection

- Token monitoring
- AI analytics
- Adaptive cards
- Voice support
- Teams bot integration

Recommended Azure Services

Azure Service	Purpose
Azure OpenAI	LLM
Azure AI Search	RAG
Azure App Service	Hosting
Azure Functions	APIs
Cosmos DB	Incident storage
Key Vault	Secret management
API Management	API security
Azure Monitor	Logging
Entra ID	Authentication

Final Enterprise Workflow

User submits issue



Incident Classifier Agent



Severity Agent



KB Search Agent



Routing Agent



Teams/Outlook Escalation



ServiceNow Ticket Creation



Dashboard Analytics

Final Outcome

This project demonstrates:

- Enterprise AI architecture
- Multi-agent systems
- Azure OpenAI integration
- RAG implementation
- AI workflow automation
- Teams/Outlook integration
- ServiceNow integration
- Enterprise governance
- Production-ready AI engineering